SEQUENCE 1

Les deux premières activités ont été extraites et adaptées du livre NSI 1re - Ed. Bordas

ACTIVITE 1 - Constructions élémentaires & langages

Adam a lu que dans la grande majorité des langages de programmation on retrouve les mêmes constructions élémentaires. Les deux premières constructions élémentaires auxquelles il s'intéresse sont les affectations et les séquences d'instructions qui sont présentes dans la fonction Python suivante:

```
1  def conversion(duree_en_minute):
2      heure = duree_en_minute // 60
3      minutes = duree_en_minute % 60
4      return heures, minutes
```

Répondre aux questions

A quelles lignes peut-on trouver:

- une affectation?
- une séquence d'instructions?

Parmi les deux fonctions suivantes, laquelle contient une boucle bornée ? une boucle non bornée ? Préciser les **mots clés** utilisés.

```
1  def estimer_quantite(quantite_initiale, annee):
2      resultat = quantite_initiale
3      while resultat <= quantite_initiale * 3:
4          resultat = resultat * 1.33
5          annee = annee + 1
6      return annee
7
8  def augmentation(quantite_initiale):
9      resultat = quantite_initiale
10     for i in range(5):
11         resultat = resultat * 1.33
12     return resultat
```

On considère maintenant les 6 fonctions suivantes.

```python
def maximum(a, b):
    if a > b:
        return a
    else:
        return b

def maximum2(a, b, c, d):
    return maximum(maximum(a, b), maximum(c, d))

def positive(valeur):
    if valeur < 0:
        valeur = -valeur
    return valeur

def conversion(en_bit):
    en_octet = en_bit / 8
    return en_octet

def racine(a):
    x = 1
    for i in range(10):
        x = (x + a / x) / 2
    return x

def compte_bits(valeur):
    nb_bits = 1
    while valeur > 2**nb_bits:
        nb_bits = nb_bits + 1
    return nb_bits
```

Répondre aux questions

Dans quelle(s) fonction(s) retrouve-t-on:

1. une séquence d'instructions ?
2. une ou des affectations ?
3. un test ?
4. une boucle bornée ?
5. une boucle non bornée ?
6. un appel de fonction ?

Adam s'attache maintenant à écrire certaines fonctions précédentes avec le langage **JavaScript** afin de les comparer à celle écrite en Python. Il commence par écrire la fonction nommée `maximum()`.

```
function maximum(a, b){
    if (a > b) {
        return a
    } else {
        return b
    }
}
```

Analyser

Citer les points communs avec le langage Python et repérer dans cet exemple 3 points particuliers du langage JavaScript.

Adam écrit à présent cette fonction sans les indentations (version 1) et sur une seule ligne (version 2).

Version 1

```
function maximum(a, b) {
if (a > b) {
return a
} else {
return b
}
}
```

Version 2

```
function maximum(a,b){ if(a>b) { return a } else { return b } }
```

Répondre aux questions

- Sachant que les deux écritures de la fonction `maximum()` sont correctes, quelles hypothèses peut-on formuler quant aux indentations et aux retours à la ligne en javascript ?
- Quel signe de ponctuation est utilisé en Javascript pour remplacer les indentations en python ?

En gardant les indentations qui améliore la lisibilité du code, Adam s'intéresse à présent à la fonction `augmentation()`.

```
1 function augmentation(quantite_initiale) {
2     let resultat = quantite_initiale
3     for(let i=0; i<5; i++) {
4         resultat = resultat * 1.33
5     }
6     return resultat
7 }
```

Analyser

- En considérant le programme JavaScript `augmentation()` ci-dessus, citer deux nouveaux points communs et repérer deux différences entre les langages Python et JavaScript.
- Formuler une hypothèse sur l'utilité d'écrire le mot clé `let` aux lignes 2 et 3.

Adam réécrit également la première fonction de cette activité, `conversion()`, en Javascript.

```
function conversion(duree_en_minute) {
    let heures = Math.floor(duree_en_minute / 60)
    let minutes = duree_en_minute % 60
    return [heures, minutes]
}
```

Répondre aux questions

- En Javascript une fonction ne peut pas renvoyer un couple de valeur (en fait un *n-uplet*). Comment Adam a-t-il choisi de renvoyer le résultat ?
- L'opérateur qui calcule le reste de la division euclidienne est identique en python et en javascript: `%`. En revanche, il n'existe pas d'opérateur en javascript pour calculer le quotient de la division euclidienne. Il faut calculer le quotient de la division et récupérer sa partie entière en faisant appel à une fonction native du langage JavaScript qui s'appelle `floor()`. Adam se rappelle avoir déjà utilisé une fonction Python qui avait le même nom. D'une manière générale, quelles difficultés peut-on rencontrer quand on utilise un nouveau langage de programmation ?

ACTIVITE 2 - Mise au point de programmes

Spécification d'une fonction

En tant que développeur python, on vous demande d'écrire une fonction de calcul de la racine carrée d'un nombre. L'entreprise dans laquelle vous travaillez demande que chaque fonction soit documentée selon le plan suivant:

1. Quel est le nom de la fonction ?
2. Combien de paramètres sont acceptés en entrée ? Préciser leur type.
3. Quel est le domaine valide des valeurs des paramètres?
4. Combien de valeurs sont renvoyées en sortie ? Préciser leur type.
5. Quel est le domaine valide de la valeur renvoyée en sortie ?

Si vos souvenirs sont bons en mathématiques, **la racine carrée** d'un nombre est toujours **positive ou nulle** et ne peut-être calculée **que pour des valeurs positives ou nulles**.

Questions

- Répondre aux questions du plan en tenant compte des informations fournies.
- Les points 1, 2 et 4 du plan forment le **prototype** de la fonction car ils définissent le minimum d'informations à connaître pour pouvoir l'utiliser. A quoi servent les points 3 et 5 ? Comment ces informations pourraient-elles être utilisées ?

Vous fournissez le programme suivant:

```python
def racine(a):
    # méthode de Héron
    assert a >= 0
    x = 1
    for i in range(10):
        x = (x + a / x) / 2
    assert x >= 0
    return x
```

Questions

- Pour chaque question du plan, indiquer la ligne du programme qui apporte la réponse.
- Tester la fonction `racine()` avec un argument invalide en entrée, comme par exemple la valeur -2. Que se passe-t-il ?
- On souhaite ignorer la ligne 3 de la fonction `racine()`. Il suffit de la commenter comme suit `# assert a >= 0`. Tester à nouveau la fonction avec la valeur -2. Que se passe-t-il ?
- Préciser alors le rôle de l'assertion utilisée à la ligne 3.

Utiliser des jeux de test

En tant que développeur python, on vous demande de vérifier le bon comportement de chaque fonction écrite. Pour cela, il faut écrire une deuxième fonction, dit de *vérification* ou de *test*.

La fonction à écrire, qui sera ensuite vérifiée, doit calculer un prix après remise: si le prix est de 100 € ou moins, il n'y a pas de réduction. Sinon, la réduction sur le prix total est de 10 €.

Vous écrivez donc les deux fonctions suivantes:

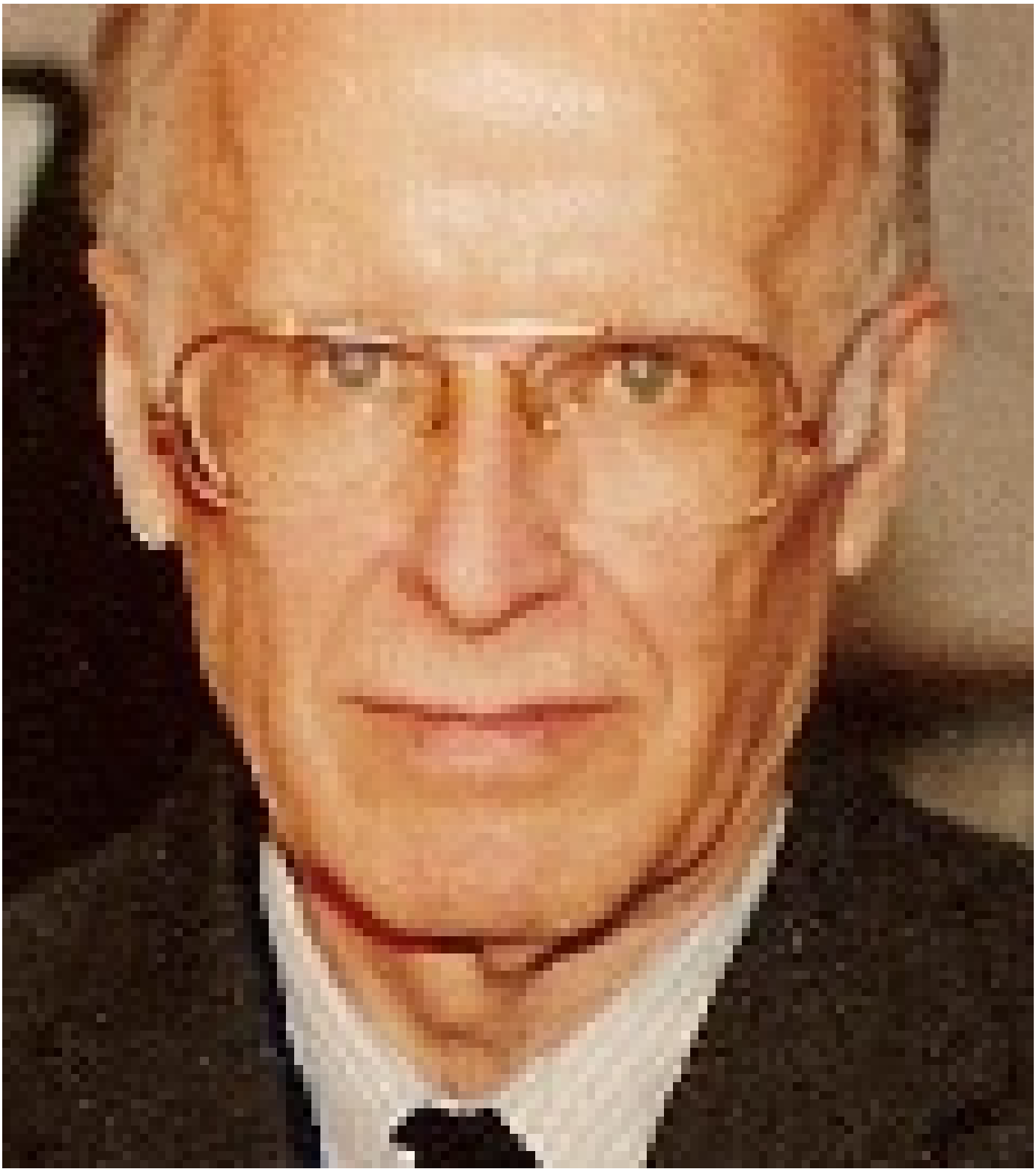
```python
def reduction(prix):
    if prix < 100:
        return prix
    else:
        return prix -10


def test_reduction():
    if not reduction(50) == 50:
        return False
    elif not reduction(150) == 140:
        return False
    else:
        return True
```

Questions

1. Si la fonction de test renvoie `False`, que peut-on en déduire sur le fonctionnement de `reduction(prix)`?
2. Que renvoie l'appel `test_reduction` ? Est-il nécessaire que le test renvoie `True` pour que la fonction soit correcte ?
3. Dans le jeu de test, quelles sont finalement les seules valeurs testées?
4. Confiant dans l'exactitude de votre fonction, vous l'intégrez dans le logiciel qui doit l'utiliser. On s'aperçoit par la suite que la réduction est appliquée pour un prix égal à 100 €. Selon vous, d'où vient le problème ?
5. La fonction de test est-elle suffisante ? Expliquer pourquoi.
6. Est-il réaliste de vouloir tester tous les cas possibles ?
7. Commenter la citation de l'algorithmicien très connu E. Dijkstra: *le test de programmes peut-être une façon très efficace de montrer la présence de bugs mais est désespérément inadéquate pour prouver leur absence*.
8. Plus tard, un autre développeur modifie votre fonction pour ajouter une réduction supplémentaire si le prix dépasse 200 €. L'appel `test_reduction()` renvoie désormais `False`. Que faut-il en déduire? Quel est l'intérêt d'avoir écrit cette fonction de test ?

COURS - Langage - Constructions élémentaires

Repère historique



En 1954, John Backus(*1924-2007*) présente le premier vrai langage de programmation tel qu'on l'entend aujourd'hui: le Fortran. Il mit au point vers la fin des années 50 le langage Algol. Il a, par la suite, proposé la *forme Backus-Naur* notation qui permet de décrire la *grammaire* des langages de programmation.

Programme et langage de programmation

Un **programme est un texte qui décrit un algorithme que l'on souhaite faire exécuter par une machine**. Ce texte est écrit dans un langage particulier, appelé **langage de programmation**. On le sauvegarde dans un fichier dont l'extension donne une indication sur le langage utilisé (`.py`, `.js`, `.c`, etc.)

Le principal langage utilisé en spécialité NSI est le langage **Python** (créé en 1991 par Guido Van Rossum) en version 3. C'est un langage de haut niveau (*d'abstraction*) et **interprété**, c'est-à-dire qu'il nécessite un autre programme appelé interpréteur qui va se charger d'exécuter ligne par ligne le *code source* du programme de l'utilisateur.

Il existe bien évidemment d'autres langages de programmation. Il s'agit ici de présenter les **constructions élémentaires** que l'on retrouve dans la quasi totalité d'entre eux.

Les entrées/sorties

Dans un langage de programmation on appelle *entrées/sorties* des constructions permettant une communication avec l'utilisateur, donnant ainsi un aspect interactif au programme.
En python, les fonctions `print()` et `input()` remplissent ces taches.

```
# Exemples
print("Hello World!")
print("Python version 3.", "Environnement", "Jupyter Notebook 5.7")
print("Python version 3.", "Environnement", "Jupyter Notebook 5.7", sep=" :: ", end=" ")
print(" !!! Pas de retour à la ligne.")
```

```
Hello World!
Python version 3. Environnement Jupyter Notebook 5.7
Python version 3. :: Environnement :: Jupyter Notebook 5.7  !!! Pas de retour à la ligne.
```

Remarque

Les arguments optionnels comme `sep` ou `end` par exemple, permettent d'adapter l'affichage aux besoins.

Un effet de la fonction `input()` est d'interrompre le déroulement du programme et d'attendre que l'utilisateur tape quelque chose au clavier. Les caractères entrés sont généralement récupérés dans une **variable**.

```
niveau = input("Quel est votre niveau en programmation (novice/intermédiaire/avancé)? ")
moyenne = input("Quelle était votre moyenne annuelle en math ? ")
print("Vous avez répondu niveau: ", niveau,"avec ", moyenne, "en math!")
```

```
Quel est votre niveau en programmation (novice/intermédiaire/avancé)? avancé
Quelle était votre moyenne annuelle en math ? 20
Vous avez répondu niveau:  avancé avec  20 en math!
```

Remarques

- `input()` **renvoie toujours une chaine de caractères** (même si l'utilisateur entre un nombre!);
- la syntaxe employée à l'intérieur de la fonction `print()` bien que correcte, sera adaptée dans les cours ultérieurs tenant compte des dernières évolutions du langage.

La déclaration-affectation

Pour pouvoir accéder aux données, un programme d'ordinateur fait abondamment usage de **variables**. En python, **une variable est une référence vers un objet mémoire** (voir video).

Contrairement à certains langages, les variables n'ont pas besoin d'être déclarées au préalable, en python. La **déclaration et l'affectation de variables** peuvent se faire en une seule instruction. En python cette opération est réalisée selon le schéma `nom_variable = valeur`.

Exemples

```
n = 7
msg = "Affectation"
pi = 3.14
```

Remarques

- le nom de variable doit commencer par une lettre et ne pas être un mot réservé (voir liste).
- Le **type** de l'objet référencé par une variable est déterminé lors de l'affectation, de manière tout à fait transparente. Dans l'exemple précédent, `n` référence un entier (`int`), `msg` référence une chaîne de caractères (`str`), `pi` référence un nombre décimal (`float`).

Séquence et bloc d'instructions

On peut définir une séquence d'instructions comme un ensemble de deux instructions exécutées l'une **à la suite** de l'autre (l'ordre est important!). Par exemple, les lignes

```
x = x + 1
x = x * 2
```

constituent une séquence d'instructions (*valide à condition que la variable x ait été affectée au préalable*). Nos programmes comporteront généralement des séquences de plus de deux instructions, on parlera alors de **bloc d'instructions**.
En python, un bloc d'instructions est repéré par son **indentation**, c'est-à-dire le décalage par rapport au bloc précédent (*typiquement un multiple de 4 espaces*).

Exemple

La séquence d'instructions ci-dessous permet de calculer et d'afficher l'aire S d'un disque de rayon r (rappel: $S = \pi \times r^2$).

```
PI = 3.14
r = 5
s = PI * r**2
print(s)
```

```
78.5
```

> **Remarque**
>
> Les instructions précédentes forment un seul bloc d'instructions (elles sont toutes alignées sur la même colonne).

Le test simple

Cette construction permet d'exécuter une instruction ou un bloc d'instructions uniquement si une condition est vérifiée. En python, la syntaxe est la suivante:

```
if condition:
    bloc_instructions
```

Exemple
La construction `x % 2 == 0` (**comparaison** du reste de la division euclidienne de x par 2 avec zéro) permet de tester si x est pair.

```
n = 50
if n % 2 == 0:
    print(n, "est pair!")
    print("Sa moitié vaut ", n / 2)
```

```
50 est pair!
Sa moitié vaut  25.0
```

Pour identifier sans ambiguités les instructions appartenant au bloc vérifiant la condition, on **DOIT LES INDENTER** en python.
La condition est souvent exprimée avec des **opérateurs de comparaisons**. Voici quelques exemples:

```
x == y # x est égal à y
x != y # x est différent de y
x > y # x est plus grand que y
x < y # x est plus petit que y
x >= y # x est plus grand que, ou égal à y
x <= y # x est plus petit que, ou égal à y
```

Le test avec alternative

Dans un test avec alternative, on ajoute un bloc à traiter lorsque **la condition n'est pas vérifiée**. La syntaxe est la suivante:

```
if condition:
    bloc_instructions_SI
else:
    bloc_instructions_SINON
```

Exemple

```
# Les test alternatif en python
n = 501
if n % 2 == 0:
    print(n, "est pair!")
    print("Sa moitié vaut ", n / 2)
else:
    print(n, " est impair")
```

```
501  est impair
```

La boucle conditionnelle, non bornée

Une **boucle conditionnelle**, appelé encore **boucle while** (*en anglais*) ou **boucle tant que** (*en français*) exécute un bloc d'instructions tant qu'une condition est vérifiée.

En python, la boucle conditionnelle a la forme suivante:

```
while condition:
    bloc_instructions
```

Remarques

- la condition s'exprime généralement avec les opérateurs de comparaisons vus plus haut;
- le bloc d'instructions est indenté.

Exemple

```
tab = 8
i = 1
while i < 10: # la condition
    print(i, " x ", tab, " = ", i*tab)
    i = i + 1
```

```
1  x  8  =  8
2  x  8  =  16
3  x  8  =  24
4  x  8  =  32
5  x  8  =  40
6  x  8  =  48
7  x  8  =  56
8  x  8  =  64
9  x  8  =  72
```

ATTENTION

Il **faut que la condition ne soit plus vérifiée** à un moment donné pour assurer la **terminaison** de la boucle et éviter le bouclage infini. Ce point sera détaillé dans le cours d'algorithmique.
Dans le cas précédent, la condition est: $i < 10$. Or, i est un entier qui augmente de 1 à chaque tour de boucle. Lorsqu'on aura atteint $i = 10$, la condition ne sera plus vérifiée et on sortira de la boucle.

La boucle inconditionnelle ou *bornée*

Lorsque le **nombre d'itérations est connu d'avance** la boucle devient **inconditionnelle**. On l'appelle encore boucle bornée ou **boucle pour** en français (*for loop* en anglais).
En python, la syntaxe de la boucle `for` est la suivante:

```
for variable_compteur in sequence:
    bloc_instructions
```

Remarques

- `sequence` est une suite d'objets, comme des entiers, des caractères d'une chaine, etc, (d'autres séquences seront vus dans les prochains cours); typiquement une séquence d'entiers est obtenue avec l'objet `range(a, b)` qui fourni les entiers allant de a à $b-1$.
- Le bloc d'instructions doit être indenté.

Exemple

```
tab = 10

for i in range(1, tab): # i est la variable compteur, range(1, tab) fourni la sequence d'entiers
    print(i, " x ", tab, " = ", i*tab)
```

```
1  x  10  =  10
2  x  10  =  20
3  x  10  =  30
4  x  10  =  40
5  x  10  =  50
6  x  10  =  60
7  x  10  =  70
8  x  10  =  80
9  x  10  =  90
```

Diversité et unité des langages

Les langages de programmations sont très nombreux. Ils peuvent être classés suivant leur *style* ou **paradigme** (façon de modéliser les problèmes et les résoudre).

Plusieurs paradigmes (orienté objet ou fonctionnel) seront abordés en terminale. En 1re, on peut se limiter au style **impératif**, dans lequel les problèmes sont résolus de manière séquentielle, en procédant à des modifications de la mémoire. Il est fait usage de *variables*, de *boucles*, etc.

Passer d'un langage à un autre peut être facile à l'intérieur d'un même paradigme car les constructions sont souvent assez proches.

On peut aussi distinguer les langages suivant le fait qu'ils soient:

- interprété[1] ou compilé[2];
- à typage dynamique[3] ou statique[4];

Exemples

Le test alternatif en Javascript

```
const N = 50

if (N % 2 === 0) {
    console.log(N + " est pair")
    console.log("Sa moitié vaut " + N/2)
} else {
    console.log(N + " est impair")
}
```

Le test alternatif en C

```
#include <stdio.h>

int main(void) {
    int N = 50;

    if ((N % 2) == 0){
        printf("%d est pair\n", N);
        printf("Sa moitié vaut %d\n", N/2);
        }
    else {
        printf("%d est impair\n", N);
        }
    return 0;
}
```

COURS - Conception & Mise au point de programmes

Repère historique

John McCarthy *(1927-2011)* auteur du langage Lisp en 1958, dont la principale construction est la définition de **fonctions**. Il joua un rôle majeur dans la programmation en intelligence artificielle, écrivant un des premiers programmes jouant aux échecs.

Qu'est-ce qu'une fonction ?

Les programmeurs décomposent souvent leur programmes en *blocs* courts. Chaque bloc de code, associé à un nom, est appelé **fonction**.
Procéder ainsi, permet:

- d'éviter les répétitions et donc de *factoriser* le code;
- de faciliter la maintenance;
- de faciliter dans une certaine mesure le travail en équipe.

> **Note**
>
> D'autres termes comme sous-programme, procédure, etc. sont parfois employés. On ne les utilisera pas en spécialité NSI

Comment définir et appeler une fonction ?

La syntaxe présentée ici est celle du langage Python. On définit une fonction avec:

- le mot clé `def` suivi d'un nom;
- une paire de parenthèses à l'intérieur desquelles figurent éventuellement des paramètres;
- le caractère *deux-points* `:` qui termine la ligne de déclaration;
- un bloc de code indenté qui forme le corps de la fonction;
- le mot clé `return` (optionnel)

```
def nom_fonction(paramètre(s)):
    bloc_instructions
```

Le nom de la fonction **doit** commencer par une lettre et **ne doit pas** être un mot reservé. Ce nom sera le plus explicite possible. La fonction peut avoir 0, 1 ou plus de paramètres.

Exemple

```
def deux_puissance(n):
    p = 1
    for c in range(n):
        p = p * 2
    return p
```

Lorsqu'on exécute ce code, il ne se passe **rien**. A ce stade on a définit la fonction, il faut maintenant l'appeler pour que tout son code soit exécuté. L'appel d'une fonction consiste à écrire **son nom suivi de parenthèses ouvrantes-fermantes** à l'intérieur desquelles on place d'éventuels **arguments**.

```python
# Calcul et affichage de 2^8 et 2^16
print(deux_puissance(8))
print(deux_puissance(16))
```

```
256
65536
```

Note

- En l'absence d'instruction `return`, python renvoie *implicitement* une valeur particulière `None` qui signifie 'rien';
- une fonction peut appeler une autre fonction.

Spécifier une fonction

Qu'est-ce qu'une spécification ?

En plus d'un nom explicite, une fonction devrait être convenablement documentée. Cette documentation devra comporter les **spécifications** de la fonction.

Spécifier une fonction consiste à donner:

- la liste ordonnée et le type de ses paramètres;
- une description du résultat renvoyé ou **post-conditions**;
- les conditions ou propriétés portant sur les paramètres d'entrée ou **pré-conditions**.

En python, la documentation suit **immédiatement** la définition de la fonction et est encadrée par des triples double quotes `"""`.

Pour garantir les post-conditions ou les pré-conditions, on peut utiliser des **assertions**.

Definition

Une assertion est une proposition logique qui doit être vraie (`True` en python) sinon le programme est arrêté.

La syntaxe d'une assertion en python est:

```python
assert expression, "Message d'erreur explicite"
```

Par ailleurs, la documentation d'une fonction est en outre accessible avec la fonction native `help()`

Exemple complet

On spécifie la fonction `deux_puissance()` précédente.

```
def deux_puissance(n):
    """
    renvoie la puissance n-ième de 2;
    n: entier naturel.
    """
    # Préconditions
    assert isinstance(n, int), "Erreur: n entier"
    assert n >= 0, "Erreur: n >= 0"

    p = 1
    for c in range(n):
        p = p * 2
    return p
```

```
deux_puissance(20)
```

```
1048576
```

En cas de non respect des préconditions, le *contrat* entre le programmeur et l'utilisateur est rompu et le programme s'arrête. Par exemple, un appel `deux_puissance(2.5)` conduit à:

```
    ---------------------------------------------------------------------------

    AssertionError                            Traceback (most recent call last)

    <ipython-input-11-34ff1cbea852> in <module>
          1 # Non respect de: "n, entier naturel".
          2
    ----> 3 deux_puissance(2.5)


    <ipython-input-9-5988c4a4a8b7> in deux_puissance(n)
          5     """
          6     # Préconditions
    ----> 7     assert isinstance(n, int), "Erreur: n entier"
          8     assert n >= 0, "Erreur: n >= 0"
          9


    AssertionError: Erreur: n entier
```

L'accès à la documentation se fait avec `help(deux_puissance)`.

```
    Help on function deux_puissance in module __main__:

    deux_puissance(n)
        renvoie la puissance n-ième de 2;
        n: entier naturel.
```

Utiliser des bibliothèques

La plupart des langages de programmation proposent des fonctions toutes prêtes. Ces fonctions fournies avec le langage sont regroupées au sein d'une **bibliothèque standard**. Il s'agit d'un ensemble de modules spécialisés (par exemple *math* ou *random*).

Pour pouvoir utiliser les objets (*des fonctions par exemple*) d'un module, il faut les **importer** avec l'instruction `import`. On privilégiera la notation `from module import objet` au lieu de `from module import *`.

Exemple

```python
# Utilisation de la fonction racine carrée (sqrt) du module math
from math import sqrt

print(sqrt(2))
```

```
1.4142135623730951
```

Une autre construction possible et aussi valable que la précédente, consiste à importer le module avec `import module` puis à utiliser l'objet avec la notation *pointée* `module.objet`.

Exemple

```python
import math

print(math.sqrt(2))
```

```
1.4142135623730951
```

Tester son code

Pour s'assurer que le programme fonctionne comme prévu, il est nécessaire de le tester. Un jeu de tests devrait couvrir les cas d'utilisation:

- typiques;
- limites.

On peut utiliser des assertions dans un premier temps (l'utilisation d'un module spécialisé peut être envisagé en python).

Enfin, une approche moderne consiste à écrire ses tests avant les fonctions.

Exemple

```
# Jeu de tests

def test_puissance():
    # Cas limite
    assert deux_puissance(0) == 1
    # Cas typique
    assert deux_puissance(10) == 1024
    return 'Tests OK'

test_puissance()
```

```
'Tests OK'
```

IMPORTANT

Le succès d'un jeu de test ne garantit pas que le programme ne contient pas de bugs. Il s'agit d'une condition nécessaire mais non suffisante.

EXERCICES Séquence 1

Exercice 1: quelques pièges de l'affectation

On définit la fonction `calcul_v1()` suivante:

```
def calcul_v1(n):
    resultat = 0
    for i in range(n):
        resultat = resultat + 1
    return resultat
```

1. Donner deux constructions élémentaires différentes ainsi que le numéro de la ligne où elles apparaissent.
2. On modifie légèrement la fonction précédente, pour obtenir une autre version:

```
def calcul_v2(n):
    for i in range(n):
        resultat = resultat + 1
    return resultat
```

L'exécution de `calcul_v2(0)` provoque cette fois-ci l'erreur suivante:

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
<ipython-input-5-e000315c4ef2> in <module>
----> 1 calcul_v2(0)

<ipython-input-4-012f1c01c96b> in calcul_v2(n)
      2     for i in range(n):
      3         resultat = resultat + 1
----> 4     return resultat

UnboundLocalError: local variable 'resultat' referenced before assignment
```

En comparant avec la première version, donner l'origine cette erreur. Proposer une explication de la situation.

1. Un élève exécute la séquence d'instructions suivante:

```
calcul_v1(100)
somme_100 = resultat
```

Malheureusement cette séquence échoue. L'interpréteur python affiche l'erreur

```
NameError: name 'resultat' is not defined
```

Cette erreur semble curieuse à priori car la variable `resultat` a été déclarée et affectée dans la fonction `calcul_v1`.

Faire une hypothèse sur la *visibilité* d'une variable déclarée à l'intérieur d'une fonction.

Exercice 2: des étoiles en boucle

Préambule: compléments sur les chaînes de caractère

Utilisation des opérateurs `+` *et* `*`

```
ch = '_/\_'
print(ch)
```

```
_/\_
```

L'opérateur `+` permet de concaténer deux chaînes.

```
print(ch + ch + ch)
```

```
_/\__/\__/\_
```

On peut aussi utiliser l'opérateur `*` entre une chaîne et un entier.

```
ch = ch * 3
print(ch)
```

```
_/\__/\__/\_
```

Dans ce dernier exemple `ch` référence un nouvel objet résultant de la concaténation `_/\_` + `_/\_` + `_/\_`.

Le caractère d'échappement `\`

Sa présence dans une chaîne modifie le caractère qui le suit. Ainsi `\n` ne désigne pas `n` mais un saut de ligne.

```
print('100\n200\n300')
```

```
100
200
300
```

Enoncé

On considère la fonction `etoilef` suivante, dont la documentation est incomplète.

```
def etoilef(n):
    """
    A compléter
    """
    a = ''
    for i in range(1, n+1):
        a = a + '*' * i + '\n'
    return a
```

1. Que réalise cette fonction? En cas de difficulté, on peut réaliser quelques appels `print(etoilef(n))`, `n` étant un entier naturel supérieur ou égal à 1.
2. Compléter la documentation de la fonction.
3. Sur la i-ème ligne, i étant un entier supérieur à 1, combien a-t-on d'étoiles ?
4. Transformer la fonction `etoilef` en une fonction `etoilew` qui utilise une boucle conditionnelle (`while`) et qui répond aux mêmes spécifications.

Exercice 3: diversité des langages

Aucune connaissance des langages cités ci-dessous n'est nécessaire pour résoudre l'exercice. Il s'agit d'un travail sur les similitudes et les différences existant entre ces langages.

Le mélange de Fisher-Yates est un algorithme qui permet de mélanger des objets (voir sa description sur Wikipedia). On se propose de comparer la traduction cet algorithme dans divers langages de programmation.

`Pascal (1970)`

```pascal
procedure shuffleList(var tab: tableau);
var
    i, j, tmp : integer;
begin
    for i := 1 to High(tab) - Low(tab) do begin
        j := random(i + 1);
        if j < i then
            tmp := tab[i + Low(tab)];
            tab[i + Low(tab)] := tab[j + Low(tab)];
            tab[j + Low(tab)] := tmp
        end;
    end;
end;
```

`C (1972)`

```c
void shuffle(int tab[], int n) {
    int i;
    for (i = 1; i < n; ++i) {
        int j = random(i + 1);
        if (j < i) {
            int tmp = tab[i];
            tab[i] = tab[j];
            tab[j] = tmp;
        }
    }
}
```

`Go (2009)`

```go
func shuffle(tab []int) {
    for i := 1; i < len(tab); i++ {
        j := rand.Intn(i + 1)
        if j < i {
            tmp := tab[i]
            tab[i] := tab[j]
            tab[j] := tmp
        }
    }
}
```

`ECMAScript (Javascript) ES 6 (2015)`

```javascript
const shuffle = (tab) => {
for (let i = 1; i < tab.length; i++) {
    let j = Math.floor((i + 1) * Math.random())
    if (j < i) {
        let temp = tab[j]
        tab[j] = tab[i]
        tab[i] = temp
        }
    }
}
```

`Python (1991)`

```python
def shuffle(tab):
    for i in range(1, len(tab)):
        j = random.randint(0, i)
        if j < i:
            tab[i], tab[j] = tab[j], tab[i]
```

1. Comment sont délimités les blocs d'instructions ?
2. Est-il nécessaire de déclarer les variables *locales* dans une fonction ?
3. Comment est structurée une boucle `for` ?
4. A partir des informations proposées, de quel(s) langage(s), le `Go` est-il proche ?

Réponses

	Pascal	C	Go	Python	Javascript (ES 6)
Délimiteurs					
Variables locales					
Structure boucle for					

Exercice 4: spécifier une fonction et tests

Lucas a déjà eu 3 notes sur 20 en NSI:

- 14 coef. 1
- 15 coef. 2
- 16 coef. 2

La dernière évaluation prévue sera affectée d'un coefficient 5. Lucas veut prévoir sa moyenne trimestrielle en écrivant une fonction `moyenne` qui prendra en paramètre sa dernière note et qui renvoie sa moyenne trimestrielle.

1. Ecrire les spécifications de la fonction `moyenne`.
2. Donner une assertion qui permet de garantir la précondition.
3. Lucas a bien pris note des bonnes pratiques de programmation. Il commence par écrire une fonction de tests de sa fonction `moyenne` :

```
def test_moyenne():
    # Cas limites
    assert moyenne(0) == ...
    assert moyenne(20) == ...
    # Cas typique
    assert moyenne(10) == ...
    return 'Tests OK'
```

   Compléter cette fonction de test.

4. Ecrire la fonction `moyenne` et tester la.

Exercice 5: divisibilité

Un moyen simple pour savoir si un entier naturel a est divisible par un entier naturel k non nul, est de tester le reste de la division de a par k. S'il est nul alors a est divisible par k. La fonction `est_divisible` ci-dessous code cette propriété.

```
1  def est_divisible(a, k):
2      """
3      Renvoie vrai (True) si a est divisible par k, faux (False) sinon.
4      a: entier naturel
5      k: entier naturel strictement positif
6      """
7      assert isinstance(a, int), "Erreur, a entier"
8      assert a >= 0, "Erreur, a entier naturel"
9      assert isinstance(k, int), "Erreur, k entier"
10     assert k > 0, "Erreur, k strictement positif"
11
12     return a % k == 0
13
14 def test_divisible():
15     assert est_divisible(0, 1) is True
16     assert est_divisible(2, 1) is True
17     assert est_divisible(10, 3) is False
18     return 'Tests OK'
```

1. Combien de paramètres requiert la fonction `est_divisible` ?
2. Quel est l'intérêt des lignes 7 à 10 ?
3. Tester la fonction `est_divisible`.

4. Prévoir le résultat renvoyé par l'appel `test_divisible()` si on rajoute avant la ligne `return ...` l'instruction `assert est_divisible(10, 0) is False`.

Exercice 6: année bissextile

Dans le calendrier grégorien, une année est bissextile si (voir article wikipedia) on se trouve dans un des cas suivants:

- l'année est divisible par 4 et non divisible par 100 ;
- l'année est divisible par 400

On souhaite coder une fonction `est_bissextile` qui prend en paramètre une année et qui renvoie le résultat logique vrai ou faux (`True` ou `False` en python) selon que l'année est bissextile ou non. Pour simplifier, on considère que le passage au calendrier grégorien a eu lieu en 1583.

1. Compléter les spécifications de la fonction `est_bissextile` ci-dessous. Seuls les `...` sont à compléter. Ne pas supprimer l'instruction `pass` qui sera remplacée par votre code plus tard.

```python
# A compléter

def est_bissextile(...):
    """
    ...
    """
    pass
```

2. Ecrire une fonction de tests `test_bissextile` en utilisant les résultats disponibles sur la page wikipedia.

```python
# A compléter après avoir supprimer l'instruction pass

def test_bissextile():
    pass
```

3. Compléter la fonction `est_bissextile` en remplaçant l'instruction `pass` par votre code et en ré-utilisant la fonction `est_divisble` de l'exercice précédent.
4. Tester votre fonction.

Exercice 7: préparer un argumentaire

Pourquoi existe-t-il plus de 730 langages de programmation dont certains sont plus utilisés que d'autres?

---

1. un programme annexe lit le code source et exécute les instructions; c'est le cas du langage Python (l'interpréteur le plus courant s'appelle CPython). ↩

2. un programme annexe transforme le code source en langage machine (une fois) avant son exécution. ↵
3. une variable n'a pas besoin d'être déclarée et peut changer de type au cours du programme (exemple en python). ↵
4. les variables doivent être déclarées et ne peuvent changer de type. ↵